

INFORME TÉCNICO

Contenedor Docker para el Módulo SGET

Sistema Inteligente de Gestión de Tráfico

Asignatura	Sistemas Operativos / Virtualización y Shell
Unidad	Contenedores, seguridad y automatización
Fecha	Mayo 2025
Autor	Estudiante

1. Introducción y Contextualización

Los contenedores se han consolidado como herramienta fundamental para el despliegue rápido, portátil y seguro de aplicaciones modernas. A diferencia de las máquinas virtuales, un contenedor comparte el kernel del sistema operativo anfitrión y empaqueta únicamente las bibliotecas y dependencias necesarias para su funcionamiento. Esto los hace significativamente más ligeros y eficientes en términos de recursos.

En el contexto del Sistema Inteligente de Gestión de Tráfico (SGET), el equipo debe diseñar e implementar un contenedor personalizado que simule un módulo de procesamiento de datos en tiempo real. Este módulo debe gestionar flujos de tráfico vehicular, registrar datos en una Micro-SD virtual, aplicar políticas de seguridad criptográfica y detectar comportamientos anómalos típicos de amenazas informáticas.

2. Elección de la Imagen Base: Alpine Linux

2.1 Justificación

Para el contenedor SGET se eligió **Alpine Linux 3.19** como imagen base. Esta decisión se fundamenta en tres pilares principales:

- **Seguridad:** Alpine usa musl libc y BusyBox, eliminando cientos de binarios innecesarios que representan superficie de ataque. No incluye bash ni herramientas de red avanzadas por defecto.
- **Ligereza:** La imagen base de Alpine ocupa apenas ~5 MB, frente a ~72 MB de Debian o ~29 MB de Ubuntu. Para un módulo IoT como el SGET, esto es crítico en recursos limitados.
- **Compatibilidad:** El gestor de paquetes apk permite instalar exactamente lo necesario (gnupg, openssl, python3) sin instalar dependencias superfluas.

Distribución	Tamaño base	Seguridad	Uso recomendado
Alpine 3.19 ✓	~5 MB	Muy alta	Contenedores IoT/producción
Ubuntu 22.04	~29 MB	Alta	Desarrollo, apps complejas
Debian 12	~72 MB	Alta	Servidores con compatibilidad amplia

3. Diseño del Dockerfile

El Dockerfile del módulo SGET está estructurado en capas funcionales que siguen las mejores prácticas de containerización segura:

3.1 Estructura del Dockerfile

```
FROM alpine:3.19
RUN apk add --no-cache bash gnupg openssl python3 shadow
RUN addgroup -S sgetgroup && adduser -S -G sgetgroup sget_user
RUN mkdir -p /app/logs /app/data /app/keys
RUN chmod 750 /app/logs /app/data && chmod 700 /app/keys
RUN passwd -l root
USER sget_user
CMD ["bash", "/app/scripts/main.sh"]
```

Cada instrucción tiene un propósito de seguridad específico: **adduser -S** crea un usuario de sistema sin contraseña ni directorio home de login; **passwd -l root** bloquea la cuenta root; **chmod 700 /app/keys** garantiza que solo el propietario pueda acceder al directorio de claves criptográficas.

4. Implementación de Políticas de Seguridad y Criptografía

4.1 Usuario con Privilegios Limitados

El contenedor ejecuta todos sus procesos bajo el usuario **sget_user**, que pertenece al grupo **sgetgroup**. Este usuario no tiene acceso a sudo, no puede escribir en directorios del sistema y está confinado al directorio /app. Esto implementa el principio de mínimo privilegio (Principle of Least Privilege).

4.2 Cifrado con OpenSSL AES-256-CBC

El script `cifrado.sh` implementa cifrado simétrico AES-256-CBC con PBKDF2 (100,000 iteraciones) para proteger los registros de la Micro-SD virtual. El proceso completo incluye:

- Generación de clave maestra aleatoria con `openssl rand -base64 32`
- Cifrado del archivo con `openssl enc -aes-256-cbc -pbkdf2`
- Verificación de integridad mediante SHA-256 (`sha256sum`)
- Descifrado controlado solo con la clave maestra

4.3 Protección de Directorios

El directorio `/app/keys` tiene permisos **700** (`rwX-----`), lo que significa que únicamente el propietario (`sget_user`) puede leer, escribir o listar su contenido. Esto protege las claves criptográficas ante otros procesos o usuarios que pudieran existir en el sistema.

5. Simulación de FreeRTOS en el Contenedor

5.1 Historia y Evolución de Arduino

Arduino es una plataforma de hardware y software de código abierto creada en 2005 en el Instituto IVREA (Italia) por Massimo Banzi y David Cuartielles. Su objetivo era proporcionar una herramienta accesible para estudiantes de diseño y artistas sin experiencia en electrónica. Existen más de 100 tipos de placas Arduino: Arduino Uno (ATmega328P, 32KB flash), Arduino Mega (ATmega2560, 256KB flash) y Arduino Due (ARM Cortex-M3, 512KB flash), entre los más comunes.

5.2 ¿Qué es FreeRTOS y para qué sirve?

FreeRTOS (Free Real-Time Operating System) es un sistema operativo de tiempo real de código abierto diseñado para microcontroladores con recursos limitados. Permite ejecutar múltiples **tareas** (tasks) de forma concurrente mediante un scheduler (planificador) que asigna tiempo de CPU según prioridades. Sus características clave son:

- Multitarea cooperativa y preemptiva
- Colas de mensajes, semáforos y mutexes para sincronización
- Bajo consumo de memoria (kernel de ~10KB)
- Portado a más de 40 arquitecturas, incluyendo ARM Cortex-M

5.3 Implementación en el Contenedor SGET

Dado que el contenedor corre en Linux (no en microcontrolador), se simuló el comportamiento de FreeRTOS mediante tres tareas de shell que operan en ciclos con retraso controlado (equivalente a vTaskDelay):

Tarea FreeRTOS	Equivalente en contenedor	Función
Task_LeerSensor	tarea_leer_sensor()	Genera datos de tráfico simulados
Task_EscribirSD	tarea_escribir_sd()	Escribe en Micro-SD virtual
Task_Verificar	tarea_verificar()	Lee con sd_seek() y valida datos
vTaskDelay()	sleep \$INTERVAL	Pausa entre ciclos del scheduler

6. Métodos de Manejo de Archivos (Micro-SD Virtual)

El script simulador_trafico.sh implementa cuatro funciones que replican la API de manejo de archivos en Arduino con Micro-SD:

Función Arduino	Función en Shell	Descripción
SD.open(file, mode)	sd_open()	Indica apertura del archivo; retorna el path
SD.write(data)	sd_write()	Escribe una línea al archivo con >>
SD.close()	sd_close()	Ejecuta sync para forzar escritura a disco
file.seek(pos)	sd_seek()	Lee desde la posición N con tail -n +N

7. Simulación de Amenaza: Infiltración e Intercepción

El script `amenaza_simulada.sh` simula el comportamiento de un proceso malicioso tipo **backdoor silencioso** dentro del entorno controlado del contenedor. La amenaza simula cuatro fases características de un ataque real:

- **Fase 1 - Reconocimiento:** El proceso intenta leer `/etc/passwd` para mapear usuarios del sistema.
- **Fase 2 - Escalada de privilegios:** Intenta ejecutar `sudo su` para obtener acceso root.
- **Fase 3 - Persistencia:** Intenta escribir en `/etc/crontab` para ejecutarse automáticamente.
- **Fase 4 - Exfiltración:** Intenta leer la clave maestra en `/app/keys`.

Todas las acciones son bloqueadas por las políticas del contenedor: el usuario `sget_user` no tiene `sudo`, `/app/keys` tiene permisos 700, y root está deshabilitado. El SGET registra cada intento en `/app/logs/amenaza_detectada.log` para auditoría posterior.

8. Relación con la Asignatura de Sistemas Operativos

Este proyecto integra múltiples conceptos de Sistemas Operativos:

- **Gestión de procesos:** Las tareas FreeRTOS se relacionan con la gestión de procesos/hilos de un SO. El scheduler de FreeRTOS es análogo al planificador del kernel Linux.
- **Gestión de archivos:** `sd_open/write/close` replica las llamadas al sistema `open()`, `write()`, `close()` de POSIX, base de los sistemas de archivos en Unix/Linux.
- **Seguridad y permisos:** El sistema de permisos `rwx` del contenedor refleja el modelo de seguridad de Unix: propietario, grupo, y otros.
- **Virtualización:** Docker es una forma de virtualización ligera (OS-level) que usa `cgroups` y `namespaces` del kernel Linux para aislar procesos.
- **Concurrencia:** Los scripts en segundo plano (&) demuestran manejo de procesos concurrentes, semejante a los hilos y sincronización en SO.

9. Conclusiones

El contenedor SGET implementado cumple con los requerimientos de la tarea: utiliza Alpine Linux como base segura y ligera, ejecuta un usuario sin privilegios, aplica cifrado AES-256 con OpenSSL, simula las tareas de FreeRTOS con lectura/escritura a Micro-SD virtual, y demuestra la detección y mitigación de una amenaza controlada. Todo el sistema refleja los principios de Sistemas Operativos: gestión de procesos, archivos, memoria y seguridad, aplicados en un contexto real de IoT y gestión de tráfico.

vehicular.